One-Week PINN Crash Course

Level 3 — In-depth neural-network knowledge

PyTorch only for every PINN component

Goal. Prepare interns to read, run, diagnose, and extend a small physics-informed neural network (PINN) codebase. This is a fundamentals course, not a claim that PINNs replace finite-element or finite-volume solvers.

PINN-Lab intern track

Version 1.0, July 2026

How to use this handbook

Outcome

By the end of Day 7, the learner can formulate a PINN loss, obtain first and second derivatives with `torch.autograd`, train a 1D heat-equation PINN, separate PDE/initial/boundary losses, test the result away from training points, and explain why a small scalar loss is not proof of valid physics.

Pace: 6 focused hours/day: 75 min watch/read, 75 min guided code, 120 min build, 60 min diagnostics, 30 min written quiz, 30 min review. Use Python, PyTorch, NumPy only for array interchange, and Matplotlib for plots. For the PINN itself, no DeepXDE, Modulus, TorchPhysics, SciANN, or other PINN library is allowed. A CPU is enough for the exercises.

Evidence rule. Every experiment must save: seed, equation and domain, network/activation, sample counts, loss weights, optimizer settings, individual loss curves, pointwise residual plot, BC/IC error, and an independent validation metric. Never report only the weighted total loss.

Shared setup

```
import torch
torch.set_default_dtype(torch.float64) # derivatives are often sensitive
torch.manual_seed(7)
device = "cuda" if torch.cuda.is_available() else "cpu"
print(torch.__version__, device)
```

Use a fresh environment with a current stable PyTorch build. Installation is platform-specific: https://pytorch.org/get-started/locally/.

The capstone problem

Solve the fixed-end wave equation $u_{tt} - c^2 u_{xx} = 0$ on $(x,t) \in [0,1]^2$, with $u(0,t) = u(1,t) = 0$, $u(x,0) = \sin(\pi x)$, $u_t(x,0) = 0$, and $c = 1$. Validate with $u^\star = \sin(\pi x)\cos(\pi t)$. Then infer positive c from a small set of interior observations.

Level 3 bridge: focus on constrained optimization and numerics

Audience. You understand backpropagation, initialization, optimization, regularization, and deep-learning experiments. Move quickly through Days 1–2; spend saved time on loss geometry, nondimensionalization, derivative accuracy, and validation.

Checkpoint

Preflight: derive reverse-mode differentiation for a scalar loss, explain Jacobian-vector products, and discuss activation smoothness. If comfortable, compress Days 1–2 into one day and use the freed day for the research stretch.

Level-specific stretch work.

1. Day 1: distinguish function-space bias from parameter-space optimization.
2. Day 2: compare autograd, finite differences, and analytic second derivatives; profile memory from `create_graph=True`.
3. Day 3: derive the effect of hard constraints on the residual and gradients.
4. Day 4: log per-objective parameter-gradient norms and cosine similarities.
5. Day 5: test static weighting versus a simple gradient-norm-based reweighting implemented directly in torch.
6. Day 6: nondimensionalize a heat equation with physical units; show how coordinate scaling alters derivative magnitudes.
7. Day 7: run an Adam-to-LBFGS schedule using `torch.optim.LBFGS`, including a correct closure, and compare wall time and validation error.

Research stretch (optional, not required for intern readiness). Read the gradient-pathology paper in full; reproduce one imbalance plot; examine spectral bias with initial data $\sin(k\pi x)$ for $k = 1, 5, 10$; write a one-page decision note on when domain decomposition, Fourier features, adaptive sampling, or a conventional solver should be considered. Do not add a third-party PINN library.

1 Day 1 — From functions to learned functions

Outcome

Explain tensors, train/validation separation, parameters, forward pass, loss, gradient descent, and why interpolation is not the same as solving physics.

Watch.

- 3Blue1Brown, "But what is a neural network?" (19 min): https://youtu.be/aircAruvnKk
- 3Blue1Brown, "Gradient descent, how neural networks learn" (21 min): https://youtu.be/IHZwWFHWa-w
- Optional calculus intuition: "Backpropagation, intuitively" (14 min): https://youtu.be/Ilg3gGewQ5U

Read.

- PyTorch, Tensors: https://docs.pytorch.org/tutorials/beginner/basics/tensorqs_tutorial.html
- PyTorch, Quickstart: https://docs.pytorch.org/tutorials/beginner/basics/quickstart_tutorial.html
- Dive into Deep Learning, MLP chapter: https://d2l.ai/chapter_multilayer-perceptrons/index.html

Practice.

1. Create tensors of shapes $(8, 1)$ and $(8, 2)$; predict the result shape of matrix products before running them.
2. Fit $y = 2x - 0.4$ with one `nn.Linear`. Write the training loop yourself; plot train loss and absolute parameter error.
3. Split 80 points into train and validation sets. Overfit 8 noisy points with a wide MLP; explain why train loss alone misleads.
4. Replace `tanh` by ReLU and sigmoid; compare convergence and output smoothness on $y = \sin(2\pi x)$.
5. Finite-difference the learned function and compare its slope with the true slope.

Debug the failure

```
model = torch.nn.Linear(1, 1)
opt = torch.optim.Adam(model.parameters(), 1e-2)
for _ in range(500):
    pred = model(x)
    loss = ((pred-y)**2).mean()
    loss.backward()
    opt.step() # BUG: what is missing?
```

Questions: Why do gradients accumulate? What pattern appears in the loss? Add `opt.zero_grad()` in the correct position and justify it.

Checkpoint

Without notes: define parameter, gradient, epoch, batch, generalization, and automatic differentiation. Why is a validation point not used to update weights?

Daily deliverable

`day1_fit.py`, one plot with train/validation curves, and a 150-word explanation of the broken loop.

2 Day 2 — MLPs, autograd, and derivatives

Outcome

Build an MLP with `nn.Module`; compute u_x, u_t, and u_{xx} with `torch.autograd.grad`; verify derivatives numerically.

Watch/read.

- 3Blue1Brown, "Backpropagation calculus" (10 min): https://youtu.be/tIeHLnjs5U8
- PyTorch, Gentle introduction to autograd: https://docs.pytorch.org/tutorials/beginner/blitz/autograd_tutorial.html
- PyTorch, neural networks: https://docs.pytorch.org/tutorials/beginner/blitz/neural_networks_tutorial.html

```
class MLP(torch.nn.Module):
    def __init__(self, widths=(2, 32, 32, 1)):
        super().__init__()
        layers = []
        for n_in, n_out in zip(widths[:-2], widths[1:-1]):
            layers += [torch.nn.Linear(n_in, n_out), torch.nn.Tanh()]
        layers += [torch.nn.Linear(widths[-2], widths[-1])]
        self.net = torch.nn.Sequential(*layers)
    def forward(self, xt):
        return self.net(xt)

def grad(outputs, inputs):
    return torch.autograd.grad(
        outputs, inputs, grad_outputs=torch.ones_like(outputs),
        create_graph=True, retain_graph=True)[0]
```

Practice.

1. For $u(x,t) = x^2 e^{-t}$, compute u_x, u_t, u_{xx} with autograd and closed form.
2. Compare autograd u_x with centered finite differences for step sizes 10^{-1} through 10^{-7}; explain truncation versus roundoff.
3. Count MLP parameters by hand and with PyTorch.
4. Use `float32` and `float64`; compare second-derivative error.
5. Run a gradient check at 50 random points and fail the test if max error exceeds a chosen tolerance.

Debug the failure

```
xt = torch.rand(100, 2) # default requires_grad=False
u = model(xt)
du = torch.autograd.grad(
    u, xt, torch.ones_like(u), create_graph=True)[0]
```

Predict the exception before running. Repair it. Then try `create_graph=False` and request a second derivative: why does that fail?

Checkpoint

Why is a smooth activation such as `tanh` a safer baseline than ReLU for a PDE containing u_{xx}? What does `create_graph=True` retain?

Daily deliverable

`day2_derivatives.py`, a derivative-error table, and two assertions that catch bad gradients.

3 Day 3 — Differential equations become losses

Outcome

Distinguish ODE/PDE, domain, residual, initial condition (IC), Dirichlet/Neumann boundary conditions (BCs), forward problem, and inverse problem.

Watch/read.

- Beginner PINN explanation and from-scratch code (about 30 min): https://youtu.be/1AyAia_NZhQ
- Raissi, Perdikaris, Karniadakis, original PINN paper: https://arxiv.org/abs/1711.10561
- Ben Moseley's PINN tutorial: https://benmoseley.blog/my-research/so-what-is-a-physics-informed-neural-network

Physics primer — Newton cooling. For temperature difference $\theta = T - T_{\text{room}}$, approximate heat loss is proportional to θ. Thus $\theta' = -k\theta$: the sign enforces decay and k has units of inverse time. A growing positive θ is physically suspicious.

Warm-up project: solve $\theta'(t) + \theta(t) = 0$, $\theta(0) = 1$, $t \in [0, 2]$. Validate against e^{-t}.

```
def ode_terms(model, t):
    t.requires_grad_(True)
    theta = model(t)
    theta_t = grad(theta, t)
    lf = ((theta_t + theta)**2).mean()
    t0 = torch.zeros(1, 1, device=t.device)
    lb = (model(t0) - 1.0).square().mean()
    return lf, lb
```

Practice.

1. Train with $(\lambda_f, \lambda_b) = (1, 1), (1, 10^{-4}), (1, 100)$.
2. Remove the IC completely. List the family of functions that satisfies the ODE.
3. Evaluate residual on a dense independent grid, not the training points.
4. Replace random points every epoch; compare with a fixed set.
5. Implement a hard constraint $u(x) = 1 + xN_\theta(x)$ and compare.

Debug the failure

```
# Zero makes theta_t + theta = 0 exactly.
lf = ((theta_t + theta)**2).mean()
loss = lf # IC term omitted
```

Why can the optimizer find a physically incomplete solution with nearly zero loss? State the missing information, not merely the missing code line.

Checkpoint

Does the differential equation alone define a unique solution? When is an IC a BC? What is the difference between training residual and validation residual?

Daily deliverable

One notebook/script comparing soft versus hard constraints and a table of L_f, L_b, dense-grid residual, and relative L_2 error.

4 Day 4 — Build the capstone PINN

Outcome

Implement a complete PyTorch-only PINN baseline, with separately logged PDE, initial, and boundary losses.

Physics primer — waves and inverse speed. A string balances transverse inertia against tension, giving $u_{tt} = c^2 u_{xx}$. Fixed ends are nodes. Without damping, total energy $\int [(u_t)^2 + c^2 (u_x)^2]/2 \, dx$ should remain constant. Energy drift or moving end nodes signals invalid physics or poor resolution.**Watch/read.**

- PyTorch PINN walkthrough (advection–diffusion): https://youtu.be/whXM-w7ig-I
- Simple PINN code walkthrough: https://youtu.be/1qyZaTF-MUQ
- PyTorch optimizer basics: https://docs.pytorch.org/tutorials/beginner/basics/optimization_tutorial.html

```
raw_c = torch.nn.Parameter(torch.tensor(0., device=device))
def speed():
    return torch.nn.functional.softplus(raw_c) + 1e-6
def residual(model, xt):
    xt = xt.requires_grad_(True)
    u = model(xt); du = grad(u, xt)
    ux, ut = du[:, 0:1], du[:, 1:2]
    uxx = grad(ux, xt)[:, 0:1]
    utt = grad(ut, xt)[:, 1:2]
    return utt - speed().square()*uxx
```

Practice.

1. Train the specified tanh MLP for 5,000 Adam steps.
2. Log L_f, L_b, L_0 and total loss separately every 50 steps.
3. Plot displacement, velocity, and a space–time error map.
4. Numerically integrate wave energy and plot drift.
5. Infer c from 20 observations; repeat with observations near a node.
6. Repeat three seeds; report median and range of relative L_2 error.
7. Time one training step at $N_f = 500, 2000, 8000$.

Debug the failure

```
utt = grad(ut, xt)[:, 0:1] # BUG: u_tx, not u_tt
```

Catch it with $u = x^2 + 3t^2$. Why constrain learned c to be positive?

Checkpoint

Why are displacement and velocity both needed? What should fixed ends and undamped energy do? Why do node observations contain little information about wave speed?

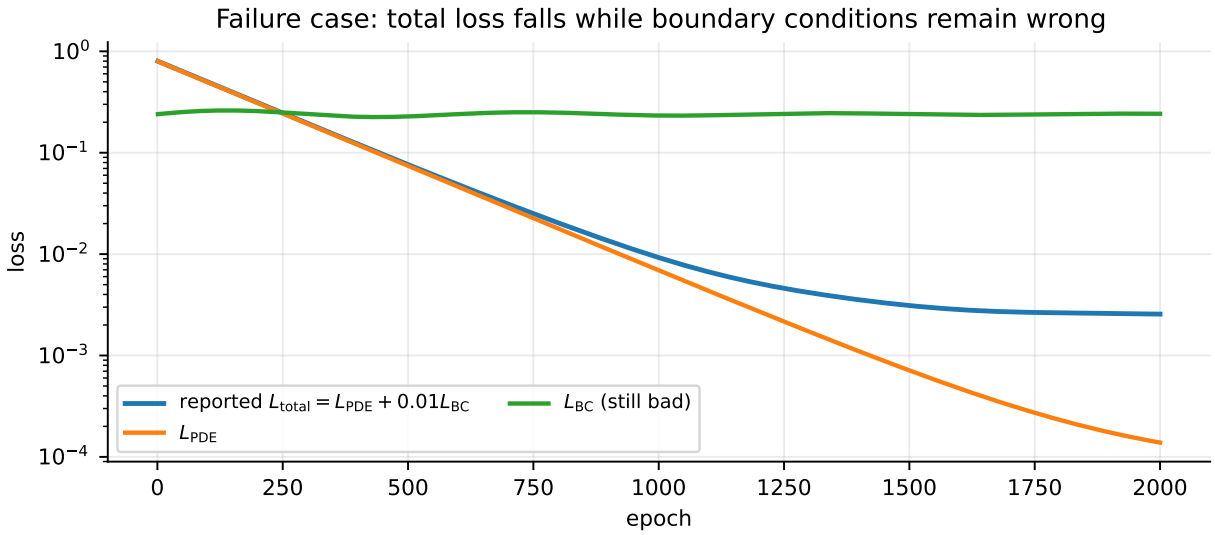
Daily deliverable

Baseline code, configuration record, four solution slices, loss-components plot, residual heat map, and seed table.

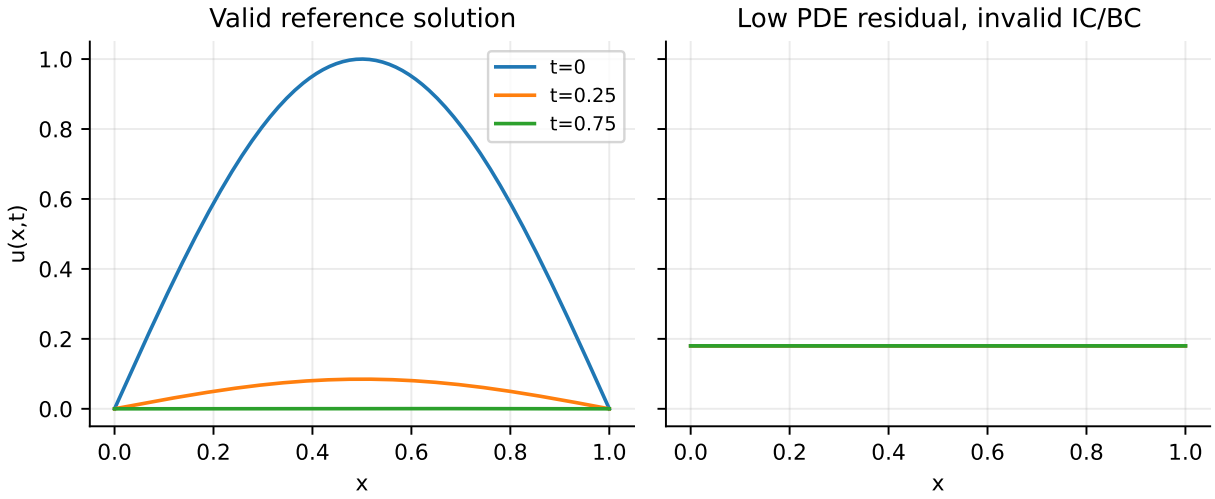
5 Day 5 — Sampling, weighting, and deceptive convergence

Outcome

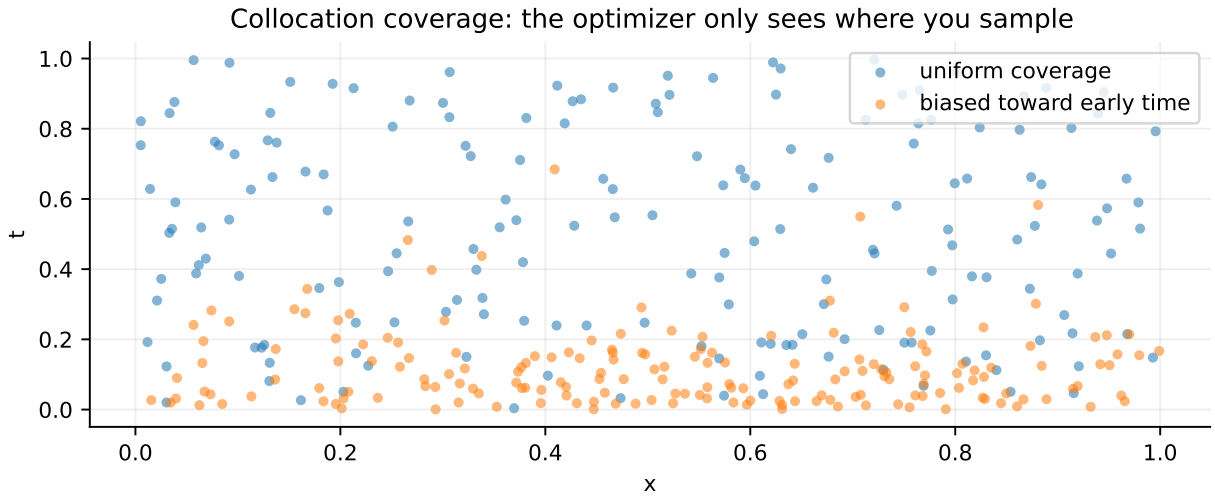
Diagnose coverage gaps and loss imbalance; demonstrate at least two cases where the reported loss decreases but the solution is unacceptable.



The weighted sum is an optimization objective, not a certificate. In the figure, L_f dominates the visual improvement while L_b remains large.



A constant field has $u_t = u_{xx} = 0$, so the heat residual is zero. It is still wrong if it violates the IC or nonzero BC data. PDE residual and well-posed constraints answer different questions.



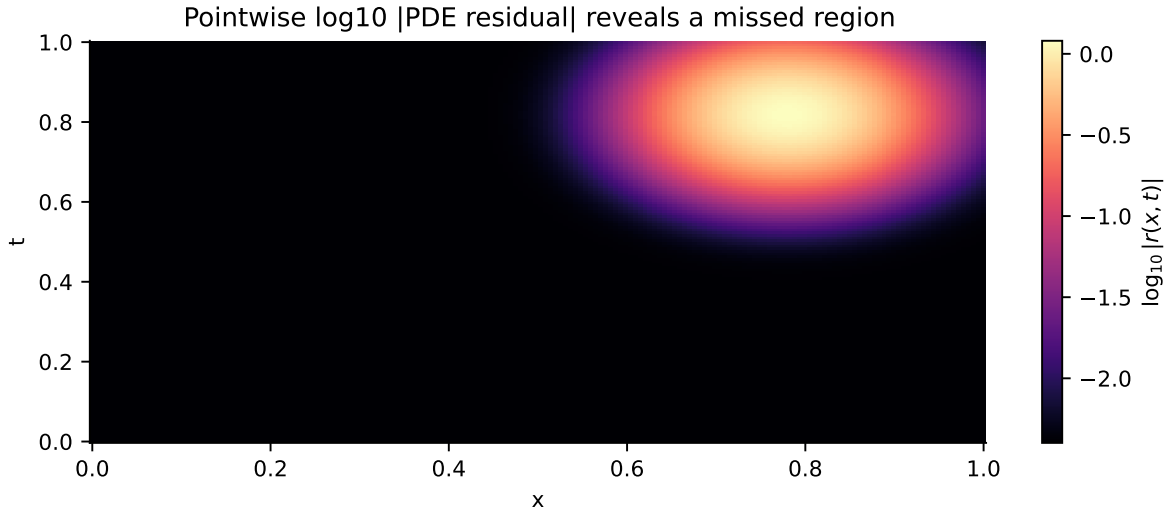
Practice.

1. Set $\lambda_b = \lambda_0 = 10^{-4}$. Confirm that total loss can look good while the boundary or initial profile is visibly wrong.
2. Sample all interior points at $t < 0.2$ but validate on the full domain. Identify the blind region.
3. Use only 20 fixed collocation points; compare training-point residual with a dense-grid maximum and 95th percentile.
4. Oversample the four corners and compare with stratified interior/edge sampling.
5. Normalize inputs from $[0, 1]^2$ to $[-1, 1]^2$; note derivative scaling.
6. Compare uniform random, a regular grid, and resampling each epoch using the same number of points.

Debug the failure

```
# 200 copies of one point are not good coverage.
x = torch.full((200, 1), 0.5)
t = torch.full((200, 1), 0.5)
xtf = torch.cat([x, t], dim=1)
```

Why is the batch size not the effective coverage? Add an assertion using the number of unique rows and a plot of the sample cloud.



Checkpoint

If mean residual is 10^{-5} and maximum residual is 1, which should be reported? Both. Explain what each reveals. Why can loss weights hide a bad term?

Daily deliverable

A "failure gallery" with two deceptive loss curves, corresponding solution plots, sample clouds, and a diagnosis/fix paragraph for each.

6 Day 6 — Verification, tests, and PINN limitations

Outcome

Treat a PINN as numerical software: use manufactured solutions, unit tests, independent grids, conservation checks, and baselines.

Read.

- Gradient pathologies in PINNs: https://arxiv.org/abs/2001.04536
- When and why PINNs fail (failure-focused paper): https://arxiv.org/abs/2207.02338
- PyTorch reproducibility: https://docs.pytorch.org/docs/stable/notes/randomness.html

Verification ladder.

1. **Shape tests:** every scalar field is $(N, 1)$; no accidental broadcast.
2. **Derivative tests:** manufactured $u = x^2 e^{-t}$ gives known derivatives.
3. **Constraint tests:** evaluate each edge and IC independently.
4. **Residual tests:** independent grid, mean/quantiles/max, and heat map.
5. **Solution tests:** relative L_2, max error, and time slices.
6. **Physics tests:** for zero-flux problems, track conserved integral; for this Dirichlet heat problem, check decay and maximum-principle behavior.
7. **Robustness:** multiple seeds, widths, point counts, and precisions.

Practice.

1. Write `pytest`-style assertions without adding a PINN library.
2. Manufacture $u = e^{-t}(x - x^2)$, compute the forcing $s = u_t - \alpha u_{xx}$, and train the forced PDE.
3. Compare PINN predictions with the analytic heat solution. If available, write a simple explicit finite-difference baseline using only torch arrays.
4. Break the sign in $u_t - \alpha u_{xx}$; decide which tests detect it.
5. Check whether the predicted maximum grows over time. Explain why that is suspicious.
6. Make a model card: equation, assumptions, domain, data, limitations, validation, failure modes, and safe-use boundary.

Debug the failure

```
# Broadcasting silently creates an (N,N) error matrix.
pred = model(init) # (N,1)
target = torch.sin(torch.pi*x0[:, 0]) # (N,)
l0 = ((pred - target)**2).mean()
```

Print shapes and explain the broadcast. Repair with `x0[:,0:1]` and add `assert pred.shape == target.shape`.

Checkpoint

Distinguish verification (did we solve the equations correctly?) from validation (are these equations a suitable model of reality?). Why is an analytic solution allowed for evaluation but not needed for PINN training?

Daily deliverable

`tests.py`, validation table, residual/error heat maps, and a one-page model card.

7 Day 7 — Capstone, code review, and handoff

Outcome

Produce a reproducible heat-equation PINN experiment and defend its validity, limitations, and next engineering step in a 10-minute review.

Capstone repository structure.

```
pinn_project/
  README.md # equation, run command, expected outputs
  config.py # seed, dtype, widths, samples, weights
  model.py # MLP only
  physics.py # derivatives and residual
  sampling.py # domain, IC, and BC samplers
  train.py # loop and separate logging
  evaluate.py # independent-grid metrics and plots
  tests.py # derivative, shape, constraint tests
  results/ # curves, maps, seed summary
```

Required experiments.

1. Baseline: tanh, float64, $(\lambda_f, \lambda_b, \lambda_0) = (1, 1, 1)$.
2. Missing-condition ablation: show the non-unique/trivial-solution failure.
3. Bad-weight ablation: make total loss misleading.
4. Bad-sampling ablation: create and expose a blind spot.
5. One repair: hard BC, reweighting, resampling, or increased coverage.
6. Three-seed evaluation on a grid never used in training.
7. Physics check: measure fixed-end error, energy drift, recovered c, and sensitivity to observation placement.

Final oral questions.

1. Write the capstone residual from memory and identify the derivative order.
2. What makes the PDE problem well posed?
3. Why does ReLU create trouble for u_{xx}?
4. How can the PDE residual be tiny for the wrong solution?
5. Why log unweighted component losses?
6. How do you prove collocation coverage?
7. Which metric would block merging this model into PINN-Lab?
8. What changes for a Neumann boundary?
9. Name two reasons PINNs can be harder to optimize than supervised NNs.
10. When would a conventional numerical solver be the better choice?

Daily deliverable

Code, README, model card, four required experiments, plots, seed table, and a 10-minute demo. Reviewers should be able to rerun the baseline with one command.

8 Level 3 graduation standard

You pass when you can defend formulation and verification, instrument competing gradient objectives, explain conditioning/scaling, and design a controlled improvement. Research novelty is explicitly out of scope.

A Curated resource map

Type	Resource and URL	Use it for
Video	3Blue1Brown neural-network playlist: https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi	intuition for NN, gradient descent, backprop
Website	PyTorch Learn the Basics: https://docs.pytorch.org/tutorials/beginner/basics/intro.html	tensors, data, model, autograd, optimization
Website	PyTorch autograd mechanics: https://docs.pytorch.org/docs/stable/notes/autograd.html	deeper graph and gradient behavior
Website	Dive into Deep Learning: https://d2l.ai/	free mathematical ML/NN text with exercises
Video	Physics-informed neural networks with PyTorch: https://youtu.be/whXM-w7ig-I	PDE-to-code walkthrough
Video	Beginner PINN from scratch: https://youtu.be/1AyAia_NZhQ	first PINN mental model
Tutorial	Ben Moseley, PINN tutorial: https://benmoseley.blog/my-research/so-what-is-a-physics-informed-neural-network/	readable ODE/PDE implementation
Paper	Original continuous-time PINN work: https://arxiv.org/abs/1711.10561	definitions and canonical examples
Paper	Gradient pathologies: https://arxiv.org/abs/2001.04536	loss imbalance and optimization failure
Paper	PINN failure analysis: https://arxiv.org/abs/2207.02338	why low residual may not imply accuracy
Project	PyTorch tutorials repository: https://github.com/pytorch/tutorials	official runnable examples
Project	Original PINNs repository (reference only): https://github.com/maziarraissi/PINNs	historical examples; do not copy framework code

Selection rule: the required path favors official PyTorch material, widely used conceptual videos, the original PINN paper, and failure-analysis readings. Interns should not spend the week comparing PINN frameworks.

B Assessment rubric (100 points)

Evidence	Points
Correct equation, domain, IC/BC formulation	15
Clear PyTorch model and autograd derivatives	15
Separate and correct loss components	15
Sampling coverage and independent validation	15
Diagnostic plots and misleading-loss failure cases	15
Tests, reproducibility, and three-seed summary	10
Reasoned limitations and conventional-solver comparison	10
Readable repository and 10-minute defense	5

Pass: 70/100 and no critical failure. Critical failures are: omitted IC/BC without explanation; derivative implementation not unit-tested; reporting only training/weighted loss; or evaluating only on training collocation points.

C Mentor answer key for failure prompts

1. **Accumulating gradients:** PyTorch adds into `.grad`; zero them before each backward pass.

2. **No `requires_grad`:** coordinate derivatives cannot be formed. Without `create_graph=True`, higher derivatives lack a graph.
3. **Missing IC:** $u' = -u$ admits Ce^{-x}; the residual cannot choose $C = 1$. For the heat equation, $u = 0$ satisfies the homogeneous PDE and BCs but not the specified IC.
4. **Wrong slice:** differentiating u_t with respect to x gives u_{tx}, not u_{xx}.
5. **Duplicate points:** a large tensor can have almost no domain coverage; check unique points, strata, and plots.
6. **Broadcasting:** $(N, 1) - (N,)$ becomes (N, N) and optimizes the wrong objective without necessarily raising an exception.
7. **Falling total loss:** weights alter visibility and gradients. Inspect raw component losses, constraint errors, pointwise residual, and solution metrics separately.

D One-page PINN review checklist

- ☐ Equation, units/scaling, domain, parameters, ICs and BCs are explicit.
- ☐ Model input/output shapes and coordinate order are documented.
- ☐ First/second derivatives pass a manufactured-function test.
- ☐ Interior, initial, and boundary samples are visualized.
- ☐ Raw L_f, L_b, L_0 are logged; weights are recorded.
- ☐ Evaluation points are independent of training points.
- ☐ Mean, quantiles, maximum residual, and spatial residual map are shown.
- ☐ BC/IC errors and solution error are shown separately.
- ☐ At least three seeds and the dtype are reported.
- ☐ A trivial solution and a sampling blind spot have been ruled out.
- ☐ Results are compared with an analytic/manufactured or numerical baseline.
- ☐ Limitations and conditions under which the model should not be used are stated.